App Dev 2 Project Report

1. Student Details

Name: RAJPUT SUMAN
Roll Number: 22f3000990
Email: 22f3000990@ds.study.iitm.ac.in

About Me: I am a student with an interest in web application development and software technologies. I enjoy learning by building practical projects and developing applications that solve real-world problems through simple and user-friendly solutions.

---

2. Project Details

Project Title: Trekking Management App

Problem Statement:

To develop a web-based application that simplifies the management of trekking activities by providing a centralized platform for **trek management, staff coordination, user registrations, trek bookings, and availability tracking**. The system aims to reduce the dependency on manual processes such as spreadsheets and phone-based coordination.

Approach:

The application was developed using **Flask for the backend and Vue.js for the frontend**, with Bootstrap used for the user interface. The system follows a role-based approach, providing separate functionalities for **Admin, Trek Staff, and Trekkers**. SQLite is used for storing application data, while Redis is implemented for caching frequently accessed trek information and Celery is used to handle background tasks such as booking-history exports and monthly activity reports. The application also includes validation to manage trek availability, prevent duplicate bookings, and restrict access according to user roles.

---

3. AI/LLM Declaration

I used **ChatGPT (GPT-5)** to assist with understanding programming concepts, debugging selected code issues, improving code structure and naming, and preparing project documentation.

The extent of AI/LLM usage was approximately **10–15%**, mainly limited to **code suggestions, debugging assistance, and documentation support**.

All final implementation, integration, testing, and project decisions were reviewed and carried out by me.

---

4. Technologies and Frameworks Used

Technology / Library Purpose
~Flask - Backend framework used to build the application's REST APIs and and server-side logic.
~Vue.js 3 - Frontend framework used to create the application's interactive user interface.
~Bootstrap 5 - Used for responsive layouts, styling, forms, buttons, tables, and other UI components.
~SQLAlchemy -ORM used to define database models and interact with the SQLite database programmatically.
~SQLite - Lightweight relational database used to store users, treks, bookings, and related application data.
~Flask-Login -Used for user authentication, session management, and maintaining logged-in user state.
~Redis - Used for caching frequently accessed trek data and as a message broker for background tasks.
~Celery - Used to execute asynchronous/background tasks such as CSV export and monthly activity report generation.
~Flask-CORS - Used to enable communication between the Vue.js frontend and Flask backend.
~Werkzeug - Used for secure password hashing and password verification.
~python-dotenv - Used to load configuration values and sensitive settings from environment variables.
Vite -Used as the frontend development and build tool for the Vue.js application.

---

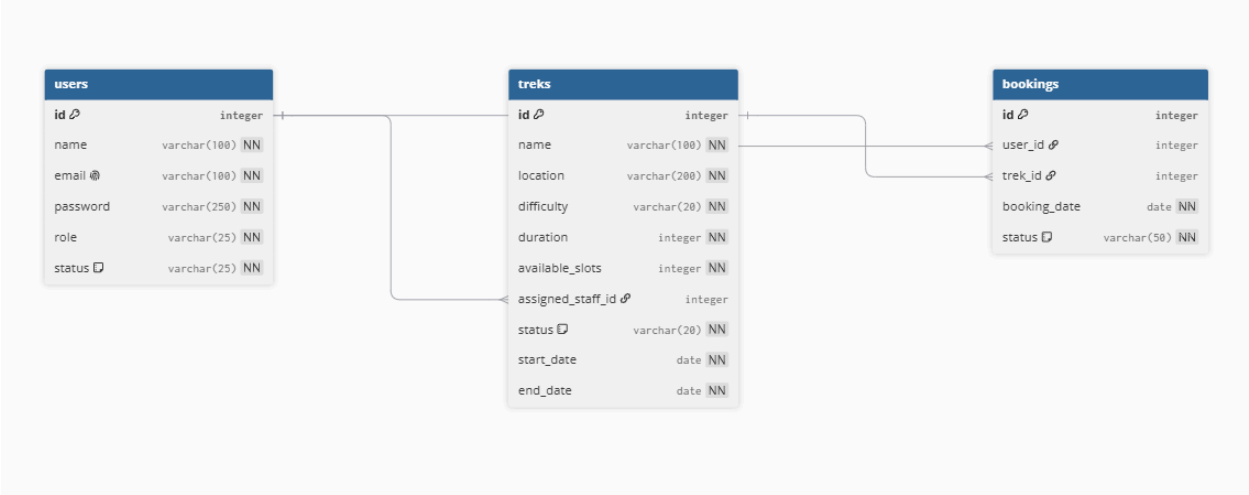
5. Database Schema / ER Diagram

Tables:

1. User — stores account and profile information for Admin, Trek Staff, and Trekkers, including user ID, name, email, password, role, and account status.
2. Trek — stores information about trekking events, including trek ID, trek name, location, difficulty, duration, available slots, assigned staff, status, and trek dates.
3. Booking — records trek bookings made by users, including booking ID, user ID, trek ID, booking date, booking status, and payment status.

Relationships:

- **One-to-Many →** `User → Booking`
- **One-to-Many →** `Trek → Booking`
- **One-to-Many →** `Staff(User) → Trek`



---

6. API Resource Endpoints

Endpoint	Method	Description
`/api/register`	POST	Register a new Trekker account.
`/api/login`	POST	Authenticate a user and create a login session.
`/api/logout`	POST	Log out the currently authenticated user.

`/api/admin/dashboard`	GET	Fetch Admin dashboard statistics.
`/api/admin/create-trek`	POST	Create a new trekking event
`/api/admin/add-staff`	POST	Add a new Trek Staff member
`/api/admin/assign-staff`	POST	Assign a staff member to a trek.
`/api/admin/treks`	GET	Retrieve and manage trekking events.
`/api/trekker/treks`	GET	Fetch available treks with search/filter options.
`/api/trekker/book/<trek_id>`	POST	Create a booking for a selected trek
`/api/trekker/my-bookings`	GET	Retrieve the user's booking and trekking history.
`/api/staff/my-treks`	GET	Fetch treks assigned to the logged-in staff member.
`/api/staff/update-trek/<trek_id>`	PUT	Update trek slots and status.
`/api/staff/complete-trek/<trek_id>`	POST	Mark an assigned trek as completed
`/api/trekker/export-bookings`	POST	Trigger an asynchronous CSV export of booking history.

---

8. Video Presentation

https://drive.google.com/drive/folders/1XBNBNuSKd_tuYddb0_dLsbGvHoW54-NE?usp=sharing